



ASSIGNMENT

Course:
Information Security

1. Honey Port

1.1 Introduction

A Honey Port is a deceptive cybersecurity technique used to detect unauthorized access attempts on a network. It involves opening fake ports or services on a system that have no legitimate use, solely to lure and trap attackers. When an attacker scans or attempts to connect to a honey port, the activity is logged and an alert is triggered, allowing security teams to identify and respond to threats.

1.2 How Honey Ports Work

The working mechanism of honey ports can be broken down into the following steps:

- A network administrator opens a specific port (e.g., port 9999) that has no real service running behind it.
- Monitoring tools or intrusion detection systems (IDS) listen on this port for any incoming connection attempts.
- When a hacker or automated scanner attempts to connect to this port, the attempt is recorded.

1.3 Key Characteristics

Port Type	Fake / Deceptive — No real service runs on it
Primary Purpose	Detection of unauthorized access and scanning activity
Implementation	Software-based trap using firewall rules or IDS
Detection Method	Any traffic to honey port is considered suspicious
Response Action	Alert generation, IP blocking, logging for forensics

1.4 Types of Honey Ports

- Low-Interaction Honey Port: Simply listens for connection attempts and logs them without engaging the attacker further.
- High-Interaction Honey Port: Engages with the attacker by simulating a real service response, gathering more detailed intelligence.
- Dynamic Honey Port: Changes the listening port frequently to capture more sophisticated scanning behavior.

1.5 Real-World Applications

Honey ports are widely deployed in enterprise networks, data centers, and cloud environments. Financial institutions use them to detect insider threats and external intrusions. They are also used in combination with SIEM (Security Information and Event Management) systems to correlate alerts and identify attack patterns automatically.

2. SSH (Secure Shell Protocol)

2.1 Introduction

SSH (Secure Shell) is a cryptographic network protocol that provides a secure channel for communication over an unsecured network. It was developed as a replacement for insecure remote login protocols such as Telnet and rsh, which transmitted data including passwords in plain text. SSH ensures that all data exchanged between client and server is encrypted, preventing eavesdropping, man-in-the-middle attacks, and data tampering.

2.2 Technical Overview

Standard Port	TCP Port 22
Protocol Version	SSHv2 (current); SSHv1 is deprecated
Encryption	AES, ChaCha20, 3DES
Key Exchange	Diffie-Hellman, ECDH
Authentication	Password, Public Key, Certificate-based
RFC Standard	RFC 4251, 4252, 4253, 4254

2.3 SSH Authentication Methods

- **Password Authentication:** The client sends a username and password, which are verified by the server. Simple but vulnerable to brute-force attacks.
- **Public Key Authentication:** The client generates a key pair (public and private). The public key is stored on the server; authentication is performed by proving possession of the private key without transmitting it.
- **Host-based Authentication:** Authentication is based on the client host rather than individual users.

2.4 Common Use Cases

- Remote server administration and command execution.
- Secure file transfer using SCP (Secure Copy Protocol) or SFTP (SSH File Transfer Protocol).
- Port forwarding and tunneling to secure other protocols.
- Accessing Git repositories securely (e.g., GitHub, GitLab).
- Automated scripts and CI/CD pipelines for deployment.

2.5 Security Features

SSH employs multiple layers of security to protect communications. It uses asymmetric cryptography for key exchange and authentication, symmetric cryptography for bulk data encryption, and message authentication codes (MACs) to verify data integrity. SSH also supports key fingerprint verification to prevent man-in-the-middle attacks during initial connection.

3. TLS (Transport Layer Security)

3.1 Introduction

Transport Layer Security (TLS) is a widely adopted security protocol designed to provide privacy and data integrity between two communicating computer applications. TLS is the successor to SSL (Secure Sockets Layer) and is the foundation of HTTPS, the secure version of HTTP used to protect web browsing, email, messaging, and many other data transfers across the internet.

3.2 TLS Version History

SSL 2.0	1995 — Original, deprecated (insecure)
SSL 3.0	1996 — Deprecated (POODLE vulnerability)
TLS 1.0	1999 — RFC 2246 (deprecated 2020)
TLS 1.1	2006 — RFC 4346 (deprecated 2020)
TLS 1.2	2008 — RFC 5246 (still widely used)
TLS 1.3	2018 — RFC 8446 (current standard, recommended)

3.3 TLS Use Cases

- HTTPS: Securing web browsing (HTTP over TLS).
- SMTPS / IMAPS / POP3S: Securing email transmission.
- FTPS: Securing file transfers.
- VoIP and messaging applications.
- API communications and microservices.
- VPN implementations using TLS/SSL tunneling.

4. Digital Certificate

4.1 Introduction

A Digital Certificate, also known as a Public Key Certificate or SSL/TLS Certificate, is an electronic document that binds a public key to an entity's identity (such as a person, organization, or website). It is issued and digitally signed by a trusted third party called a Certificate Authority (CA). Digital certificates are the cornerstone of Public Key Infrastructure (PKI) and enable secure encrypted communication across the internet.

4.2 Components of a Digital Certificate (X.509 Standard)

Version	X.509 version (v1, v2, or v3)
Serial Number	Unique identifier assigned by the CA
Signature Algorithm	Algorithm used to sign the certificate (e.g., SHA-256 with RSA)
Issuer	Name of the Certificate Authority that issued the certificate
Validity Period	Not Before and Not After dates (certificate lifespan)
Subject	Identity of the certificate owner (domain, organization, person)
Public Key	The owner's public key and the algorithm used

4.3 Types of Digital Certificates

- Domain Validated (DV) Certificate: Verifies only domain ownership. Issued quickly and used for basic encryption. E.g., Let's Encrypt certificates.
- Organization Validated (OV) Certificate: Verifies domain ownership and organizational identity. Used by businesses and organizations.
- Extended Validation (EV) Certificate: The highest level of validation, requires thorough vetting of the organization. Previously triggered the green address bar in browsers.
- Wildcard Certificate: Secures a domain and all its subdomains (e.g., *.example.com).

4.4 Certificate Authority (CA) Hierarchy

Digital certificates operate within a hierarchical trust model called the Certificate Chain or Chain of Trust:

- Root CA: The top-level CA whose certificate is self-signed and pre-installed in browsers and operating systems. Examples: DigiCert, Comodo, GlobalSign, Let's Encrypt.
- Intermediate CA: An intermediary CA certified by the Root CA. It issues end-entity certificates, protecting the Root CA's private key from exposure.
- End-Entity Certificate: The certificate issued to the final subject (website, server, user).

5. Malicious Code

5.1 Introduction

Malicious code, commonly referred to as malware (malicious software), is any code intentionally designed to cause damage, disruption, unauthorized access, or harm to computer systems, networks, or users. It represents one of the most significant threats in information security, affecting individuals, businesses, critical infrastructure, and governments worldwide. Malicious code can be delivered through email attachments, malicious websites, infected software, removable media, and network exploits.

5.2 Classification of Malicious Code

Virus	Attaches itself to legitimate programs; spreads when infected programs are executed. Can corrupt files and system data.
Worm	Self-replicating malware that spreads across networks without human interaction by exploiting vulnerabilities.
Trojan Horse	Disguises itself as legitimate software. Does not self-replicate but creates backdoors or steals data when executed.
Ransomware	Encrypts victim's files and demands ransom for decryption. E.g., WannaCry, REvil, LockBit.
Spyware	Secretly monitors user activity, captures keystrokes, screenshots, and credentials.
Adware	Delivers unwanted advertisements; may also collect browsing data without consent.

5.3 Methods of Delivery

- **Phishing Emails:** Malicious attachments or links in deceptive emails that trick users into executing malware.
- **Drive-by Downloads:** Malware automatically downloaded when a user visits a compromised or malicious website.
- **Malvertising:** Malicious code embedded in online advertisements served through legitimate ad networks.
- **Software Vulnerabilities:** Exploiting unpatched vulnerabilities in operating systems or applications.

5.4 Impact of Malicious Code

- **Data Theft:** Sensitive personal, financial, and corporate data stolen and sold on dark web markets.
- **Financial Loss:** Ransomware payments, business downtime, and recovery costs run into millions of dollars.
- **System Damage:** Corrupted files, deleted data, and permanently damaged hardware.
- **Reputational Damage:** Loss of customer trust following data breaches.
- **National Security Threats:** State-sponsored malware targeting critical infrastructure (power grids, hospitals, military systems).

5.5 Detection and Prevention Techniques

- **Antivirus and Anti-malware Software:** Signature-based and heuristic detection of known and unknown malware.
 - **Intrusion Detection Systems (IDS):** Monitor network traffic for malicious patterns and anomalies.
 - **Endpoint Detection and Response (EDR):** Advanced tools that detect, investigate, and respond to endpoint threats in real time.
 - **Sandboxing:** Executing suspicious files in an isolated environment to observe behavior safely.
-

- Regular Patching: Applying security updates to eliminate vulnerabilities exploited by malware.
 - User Awareness Training: Educating users to recognize phishing and social engineering attacks.
-